

B U P T

# 密码工程

## 第11讲 密码协议、密钥管理、密钥协商协议

授课教师：王晨宇

时间：2026年6月14日

5.1

# 密码协议

# ► 密码协议



- ✓ 5.1.1 角色
- ✓ 5.1.2 信任
- ✓ 5.1.3 动机
- ✓ 5.1.4 密码协议中的信任
- ✓ 5.1.5 消息和步骤



## 协议的参与方

- 协议一般描述为由 Alice 和 Bob 执行，或者在客户和商家之间执行。
- 像 “Alice”、“Bob”、“客户”和“商家” 这些名字并不真正表示一个具体的个体或者机构，他们代表协议中的一个角色。
- 例如，一台电脑配一个鼠标。
  - “电脑”和“鼠标”就是角色，这个角色泛指一类设备，而并不指某一个具体的型号。



## 协议的参与方

- 和上一个例子不同的是，在协议中，单个实体可以扮演任何角色。
- 在分析密码协议的安全性时，记住这一点尤为重要。
- 例如，攻击者 Eve 对协议发动中间人攻击。
  - 那么在攻击过程中，Eve 同时扮演了 Alice 和 Bob 的角色。



## 信任的概念

- 信任是人与人之间交流和互动的基础。
- 如果你不信任某个人，那么与他/她交往就显得尤为困难。
- 例如，客户需要信任商家，才能在购买商品时进行支付。
  - 商家也需要信任客户支付货款。
  - 如果没有信任，交易将无法顺利进行。



## 互联网上的信任为什么困难

- 在数字时代，互联网上的信任问题尤为突出。
- 信息不对称：
  - 网络上信息来源复杂，难以判断信息的真实性。
- 匿名性：
  - 用户身份难以验证，容易引发虚假行为。
- 网络欺诈：
  - 电商平台或支付平台上的诈骗行为会侵蚀用户的信任。



## 信任的困境与难题

- 信任不是黑与白的问题，它不是简单地“信任”或“不信任”。
- 比如，对于不同的角色，我们的信任程度会有所不同。
  - 你可能信任一个朋友，并借给他100元，但是如果是500万，可能就需要慎重考虑。
  - 我们信任银行会确保我们的存款安全，但需索要收据或存折，这说明我们无法完全信任他们的管理。
  - 信任的程度和背景密切相关，依赖于不同情境中的具体问题。





## 案例：Alice与百度

- 假设 Alice 生活在国外并连接到百度网站，她向百度请求信任。
- 但百度可能没有任何合理的理由信任 Alice 。
- 此时，如果百度采取错误的信任策略，可能会导致资金损失或信息泄露。



## 什么是动机

- 动机是协议分析中的关键组成部分。
- 不同参与者的目标是什么？他们希望从中获得什么？
- 例如：
  - 研究人员知道他们的客户想要听什么，如果他们能提出“好”的结果，就可以再次得到合同，这就会引起偏见；
  - 对于研究工作的赞助者而言，他们并不打算公布任何与自己相关的负面研究结论。



### 动机不仅仅影响协议的功能，还会改变协议的架构和设计

□ 假设有一个在线支付系统的协议：

□ 支付方希望确保交易过程中的资金安全。

□ 消费者则希望保障个人信息的隐私。

□ 为了满足这些动机，协议的结构可能会加入更强的加密措施和隐私保护机制，从而使协议能够满足双方的需求，避免信息泄露和资金损失。



### 密码协议为何需要信任管理？

- 密码协议的一个核心功能，是尽量减少需要信任的人数和信任量。
- 设计密码协议时，默认所有人都会作弊，是最稳妥的做法。每一个协议参与方都要假设其他方是潜在的攻击者。
- 例如：
  - Alice 参加一个协议，假设其他所有人可能联合欺骗她。



### 信任验证常被忽略

- 很多协议设计者会忽略验证信任假设，认为某些事情是“显然可信”的。
- 实际上，任何系统模块都应该像工程模块一样，清晰地定义边界和信任假设。
- 不验证信任，容易被攻击者利用。



## 协议中的消息和步骤

- 协议一般描述参与方之间要交换的消息内容和需要完成的操作。
- 大部分协议都只在较高层面描述，很多细节步骤被省略了。
- 虽然这样可以突出重点功能，但也带来了安全风险。

## 为什么细节很重要？

- 没有细致描述每个参与方的操作，协议实现就容易出现漏洞。
- 协议文档过于复杂也不好，没有人能100%理解，难以做到完全安全。



## 协议设计中的层次与模块化

- 解决方法：模块化设计
- 类似通信协议，把协议分成多个功能层次，每一层只处理一部分功能。
- 每一层都是建立在上一层基础上的。
  - 高层通常比较灵活可变，底层的稳定性至关重要。
- 只有明确划分各层，清晰说明每层的职责，协议才能真正做到安全可控。



## 例子：传输层

- 负责在参与方之间传递字节数据。
- 通过 UDP、TCP、Email 等手段发送信息。
- 关键问题：
  - 传输层可能需要加额外的编码，比如加长度字段，防止消息截断或篡改。
  - TCP 传输时，需要注意添加长度信息以支持流式通信。





## 传输层的挑战与设计要求

- 传输的数据应完整且不可随意修改。
- 消息传输时要保持字节串一致，避免被篡改。
- 如果在传输层已经进行了加密、认证等操作，那么整体协议的安全设计会简单很多，也能有效防止各种攻击。



## 协议和消息标识

- 接收消息时，必须知道：它属于哪个协议？是协议中的哪一条消息？
- 协议标识符：
  - 确认消息属于哪个协议（如支付协议、退款协议等）。
  - 避免不同协议之间消息混淆。
- 消息标识符：
  - 明确指出消息在协议中的具体位置。
  - 防止消息错乱，保证通信流程有序。



## 为什么要有这么多标识?

- 防止伪造攻击。虽然攻击者可以伪造，但标识可以快速检测错误。
- 良好的错误检测非常重要，有助于发现潜在攻击或配置错误。
- 标识符让消息更独立、系统更易维护。
- 设计时要遵循“越清晰越好”的原则，例如：签名时应明确是哪条消息。



## 消息编码和解析

### □ 编码：

- 消息必须被转换成字节串来进行发送。

### □ 解析：

- 接收到字节串后，要能准确还原出每个数据元素。
- 不能依赖上下文信息，必须独立解析每个字段。

### □ 为什么不能依赖上下文信息？

- 依赖上下文的信息容易出错，导致协议漏洞。
- 早期协议因为解析不规范，出过很多问题。



## 消息编码方法

□ 推荐使用TLV编码（Tag-Length-Value）：

□ Tag：标明字段类型。

□ Length：标明字段长度。

□ Value：字段实际内容。

□ 注意：

□ 避免复杂笨重的标准（如ASN.1），保持编码简单、易解析。



## 协议执行状态

- 一台计算机可以同时参与多个协议，需要跟踪每个协议的执行情况。
- 协议状态记录了当前需要等待或处理的所有消息信息。
- 如何实现？
  - 通常通过事件驱动编程，如单线程轮询、事件分派系统等方式来管理。
  - 每收到一条消息，就根据当前状态解析处理，进入下一个状态。



## 错误处理

- 协议必须进行严格的错误检测，包括：
  - 检查协议类型。
  - 消息类型是否正确。
  - 检验密码验证是否通过。
  
- 如果检测失败，应立即终止会话并清除状态，避免泄露协议内部信息。



## 错误信息带来的风险

- 错误信息可能成为攻击者获取协议细节的重要线索。
- 最好的做法：
  - 尽量不发送详细错误信息，简单回复“出现错误”即可。
- 现实案例：
  - 攻击者 Eve 可以伪造消息，引诱受害者发回错误响应，从而推测出内部状态。





## 真实案例分析：智能卡PIN攻击

□背景：智能卡系统设计允许连续输错若干次 PIN 后锁卡。

□攻击方式：

□通过观察每次输错 PIN 后的响应时间或错误提示，攻击者可以一步步推测每一位数字。

□只需少量尝试，就可以在远低于暴力破解成本的情况下推测出正确的 PIN 码。

□教训：

□不仅要限制尝试次数，还要避免在不同失败情形下有不同的响应时间和错误提示。



## 重放与重试

### □重放攻击：

- 攻击者记录一条合法消息，之后再次发送同样的消息。
- 消息看起来是正确的，但可能被恶意重复利用。

### □重试攻击：

- 通常由于收不到确认回复，导致发送方自动重发上一条消息。
- 在自动系统中，重试行为很常见，但需要正确防护，防止被攻击者利用。



## 重试的常见原因与防护

### □ 为什么会重试？

- 网络丢包、系统超时、确认信息未及时到达。

### □ 如何防护？

- 接收方需要能区分正常重试与恶意重放。

- 确保每次回应时执行正确的状态检查，避免安全漏洞。



## 重试机制与协议执行状态

- 保持一个计数器，如果超时未收到应答，就重发消息。
- 不同传输协议（如 UDP、TCP）对丢包和重发有不同的处理方式。



## 收到消息时的处理策略

□收到消息后，需要判断：

□是期望中的那条消息吗？

□如果是，按协议继续执行。

□如果是未来的消息（如提前到达的），应丢弃并忽略。

□安全做法：

□默认丢弃不在当前预期中的任何消息。



## “旧”消息的特殊处理

### □什么是旧消息？

□是指已经在之前处理过的消息。

### □应对策略：

□如果是之前响应过的消息，应给出与先前相同的回应。

□不重新计算，不生成不同的新回应，确保一致性和安全性。



### 当消息看起来正确但数据不同怎么办？

- 可能是攻击者伪造的新消息。
- 必须立即识别并按照协议判定为错误，不可被引导进入错误状态。
- 安全设计原则：
  - 任何验证失败或不符的消息，都要作为攻击处理。



**1.协议中每条消息都要包含标识符、顺序号、加密认证.....为什么这些细节反而比“功能逻辑”更重要??**



5.2

# 密钥管理



# 密钥管理

B U P T



- ✓ 5.2.1 密钥管理
- ✓ 5.2.2 密钥全生命周期
- ✓ 5.2.3 密钥生成
- ✓ 5.2.4 密钥存储
- ✓ 5.2.5 密钥导入和导出
- ✓ 5.2.6 密钥分发
- ✓ 5.2.7 密钥使用
- ✓ 5.2.8 密钥备份与恢复
- ✓ 5.2.9 密钥归档
- ✓ 5.2.10 密钥销毁



# 密钥管理

## 5.2.1 密钥管理

B U P T

密码系统的安全性不只取决于算法是否安全，更取决于密钥是否被正确的管理。

➤ 密钥管理的目的：

维持系统中各实体之间的密钥关系，以抗击各种可能的威胁：

- 密钥的泄露
- 秘密密钥或公开密钥的身份真实性丧失
- 未经授权使用

➤ 密钥管理的原则：

- 密钥难于窃取
- 密钥有使用范围和时效
- 密钥的分配和更换过程对用户透明，用户不一定要掌管密钥

➤ 密钥管理方法：因所使用的密码体制（对称密码和公钥密码体制）而异。

➤ 密钥生命周期：密钥管理是指在一种安全策略指导下密钥的**产生、存储、使用，更新，删除**等处理，涵盖了密钥的整个生命周期

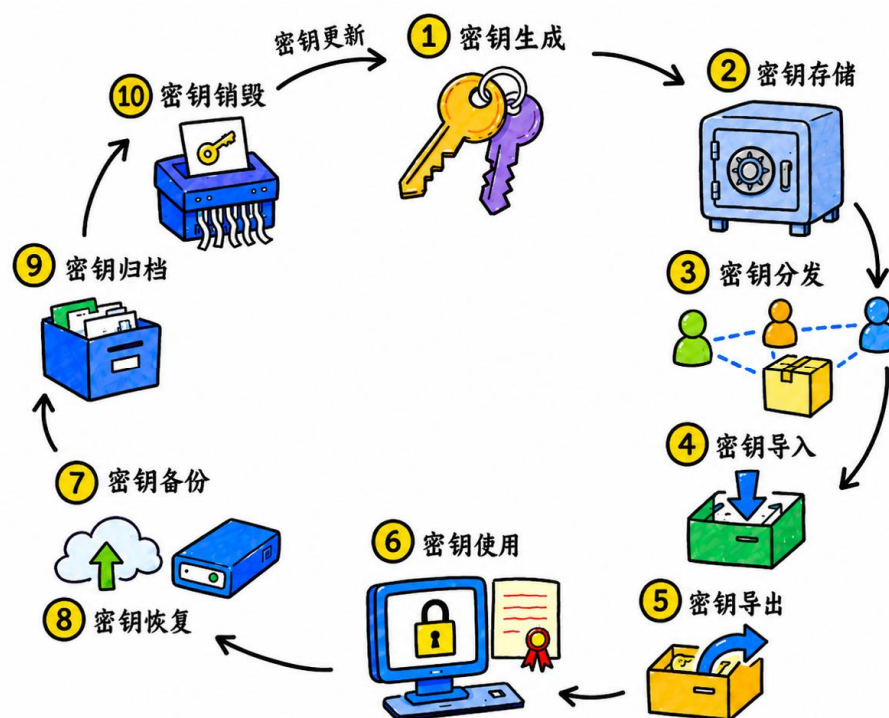


# 密钥管理

## 5.2.2 密钥生命周期

B U P T

国家密码管理局于 2021 年 10 月 1 日实施 GB/T 39786-2021《信息安全技术 信息系统密码应用基本要求》，是指导商用密码应用与安全性评估的基础性标准。政务信息系统密码应用指南也明确要求密钥管理方案覆盖密钥生成、存储、分发、导入、导出、使用、备份、恢复、归档、销毁等全生命周期。

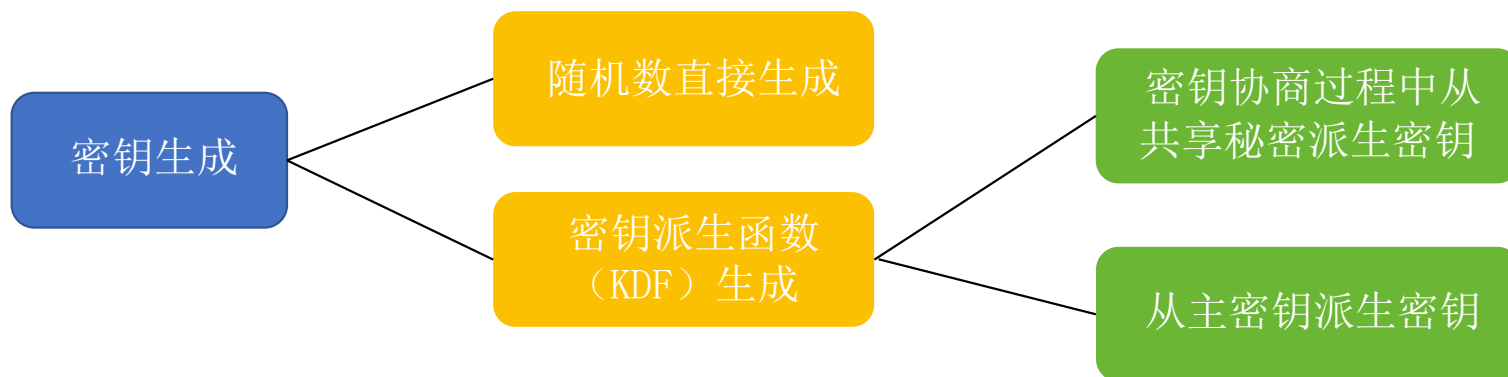




## 密钥管理 5.2.3 密钥生成

B U P T

密钥安全的第一步，是密钥必须真正随机、来源可信。



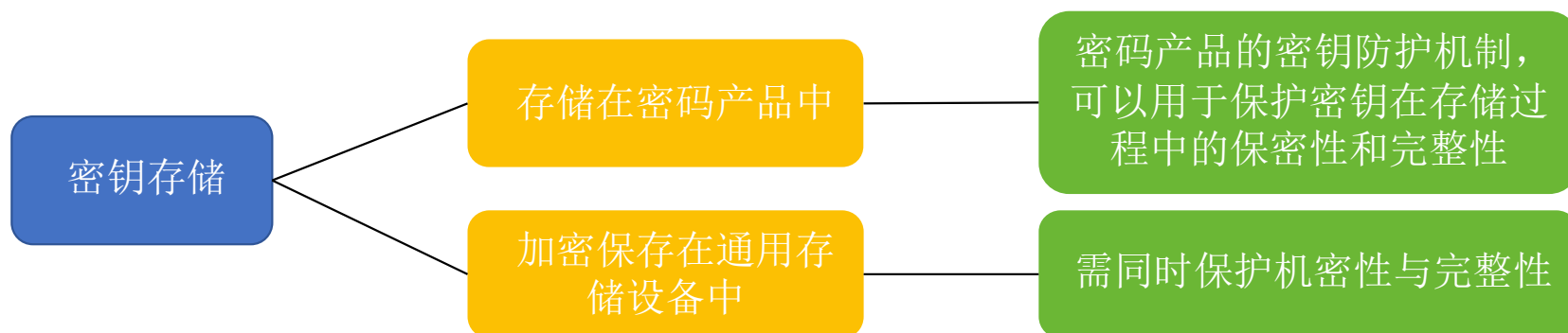
- 密钥生成依赖于高质量的随机数发生器，需使用国家密码主管部门批准或认证的设备，确保密钥的安全性和质量。在产品使用或特定场景中，通过密钥派生函数从主密钥派生出其他密钥，或在密钥协商过程中从共享密钥派生出新密钥，主密钥必须通过随机数直接生成。
- 对于需要生成大量对称密钥的场景，如大规模智能C卡分发，发卡方保存主密钥，根据实体唯一标识和其他相关信息，派生出每个实体的单独密钥，解决密钥存储和管理问题。
- 密钥派生函数在密钥管理中扮演重要角色，通过实体的唯一标识和其他信息，从主密钥派生出每个实体的单独密钥，实现高效、安全的密钥分发和管理。



## 密钥管理 5.2.4 密钥存储

B U P T

密钥存储不能明文暴露，也不能因为过度保护导致业务不可用。

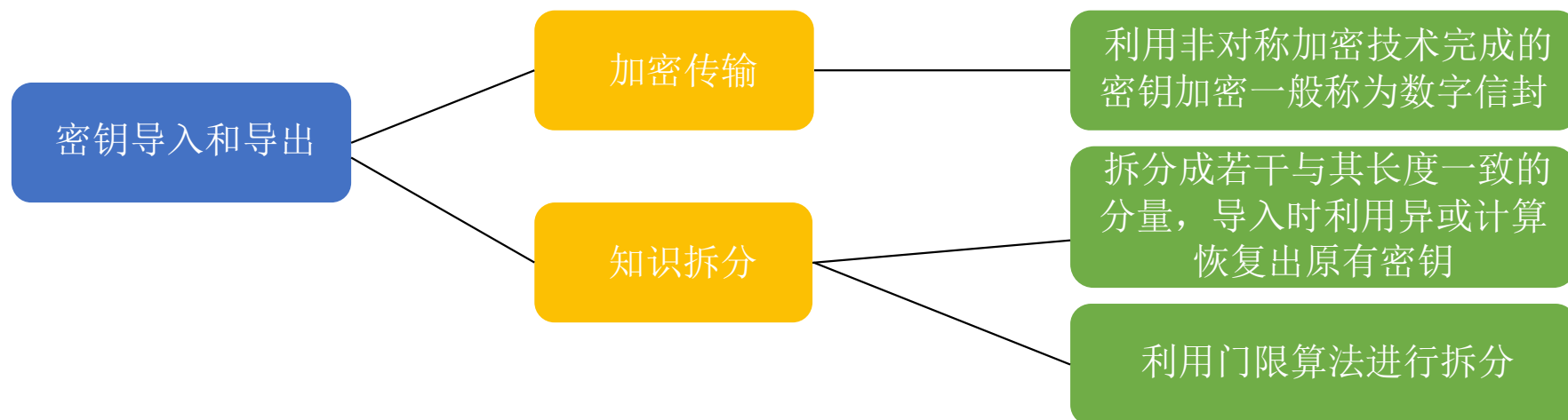


- 密钥存储在密码产品中时，应检查密码产品是否具备认证证书及相应的安全等级，确保密钥在存储过程中的机密性和完整性得到保护。密钥分层保护机制中，下层密钥利用上层密钥加密，顶层密钥加密密钥需采取严格防护措施，避免非授权获取或篡改。
- 当密钥加密保存在通用存储设备中，如数据库或服务器，需评估其机密性和完整性保护措施是否合理有效。评估密钥存储安全性时，重点在于确认保护措施合理性和执行的有效性，确保密钥安全。密钥存储的不安全性常在考题中暗示，需根据存储方式(密码产品或通用设备)采取相应安全措施和检查点。



## 密钥管理 5.2.5 密钥的导入和导出

B U P T



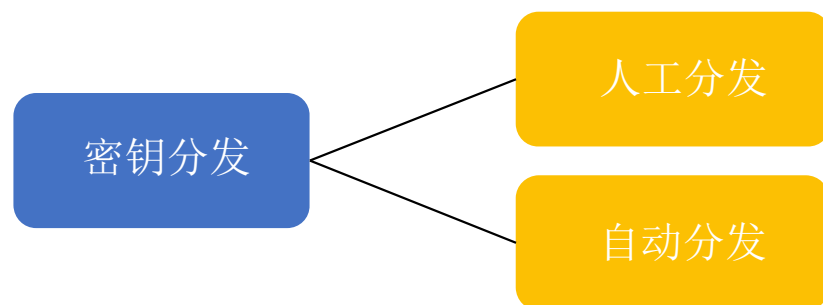
- 密钥的导入和导出主要通过两种方式:非对称加密技术(数字信封)和知识拆分，后者需确保不降低密钥安全性。
- 知识拆分需避免简单截取密钥为若干段，如将128位SM4密钥拆分为2个64比特，这种做法会降低安全性。对于高安全等级的密码产品，拆分后的密钥分量需通过可信信道传输，以保障安全。良好的知识拆分不应减少密钥位数，例如，将128位密钥拆分为2个128位片段，并通过计算恢复，确保安全性不降低。在评估知识拆分的安全性时，关键在于保持密钥位数不变，只要位数未减少，即可认为拆分过程是安全的。



## 密钥管理 5.2.6 密钥分发

B U P T

密钥分发是密钥从一个实体安全传递到另一个实体的过程。



- 密钥分发是不同密码产品间共享密钥的关键过程，主要分为人工和自动两种方式，其中人工分发依赖线下面对面等传统手段完成密钥的安全传输。自动分发利用密码技术在线完成密钥分发，安全性主要由密码技术本身保证，实现了密钥分发的高效与安全。
- 人工分发与自动分发的主要区别在于是否需要人工参与，前者需要人工介入完成密钥的线下安全分发，后者则完全在线上通过密码技术自动完成。自动分发方式借助现代密码学技术，不仅提高了密钥分发的效率，还增强了整个过程的安全性，是现代密码学领域的重要应用之一。
- 密钥分发的两种方式各有优势，人工分发在特定场景下仍具有不可替代性，而自动分发则代表了未来密钥管理的发展趋势，特别是在大规模网络环境中。





- 用于核准的密码算法的密钥，不能再被非核准的密码算法使用，因为这些算法可能导致密钥泄露。
- 特别是，不同类型的密钥不能混用，一个密钥不能用于不同用途(加密、签名、MAC等)
- 虽然不需要保护公钥的保密性，但在使用前（如签名验证或者密钥协商过程）需要验证公钥的完整性，以及实体与公钥的关联关系，以确保公钥来源的真实性。

密钥使用必须遵守最小用途原则：一个密钥只服务于一个明确的密码目的！



- 密钥备份与密钥存储非常类似，只不过备份的密钥处于不激活状态(即不能直接用于密码计算)，只有完成恢复后才可以激活。
- 密钥备份需要保护备份密钥的保密性、完整性及其与拥有者身份和其他信息的关联关系。
- 密钥备份或恢复时应进行记录，并生成审计信息;审计信息应包括备份或恢复的主体、备份或恢复的时间等。



- 密钥在其生命周期结束时，应当进行销毁。但是出于解密历史数据和验证历史签名的需要，有些不在生命周期内的密钥可能需要持续保存，需要注意的是**签名密钥对的私钥不应进行归档**。
- 密钥归档时，也应当继续对这些密钥提供安全保护，以保证历史加密数据的安全性。密钥归档时应进行记录，并生成审计信息:审计信息应包括归档的密钥和归档时间等。



- **正常销毁:**密钥在设计的使用截止时间时自动进行销毁，比如，临时密钥在使用完毕时应当立即销毁。
- **应急销毁:**密钥在达到泄露或存在泄露风险时进行的密钥销毁。对于存储在密码产品中的密钥，一般配备了紧急情况下自动销毁密钥的机制;当密钥所有者发现密钥存在泄露风险时，可能需要手动提前终止密钥的生命周期，进行密钥销毁。

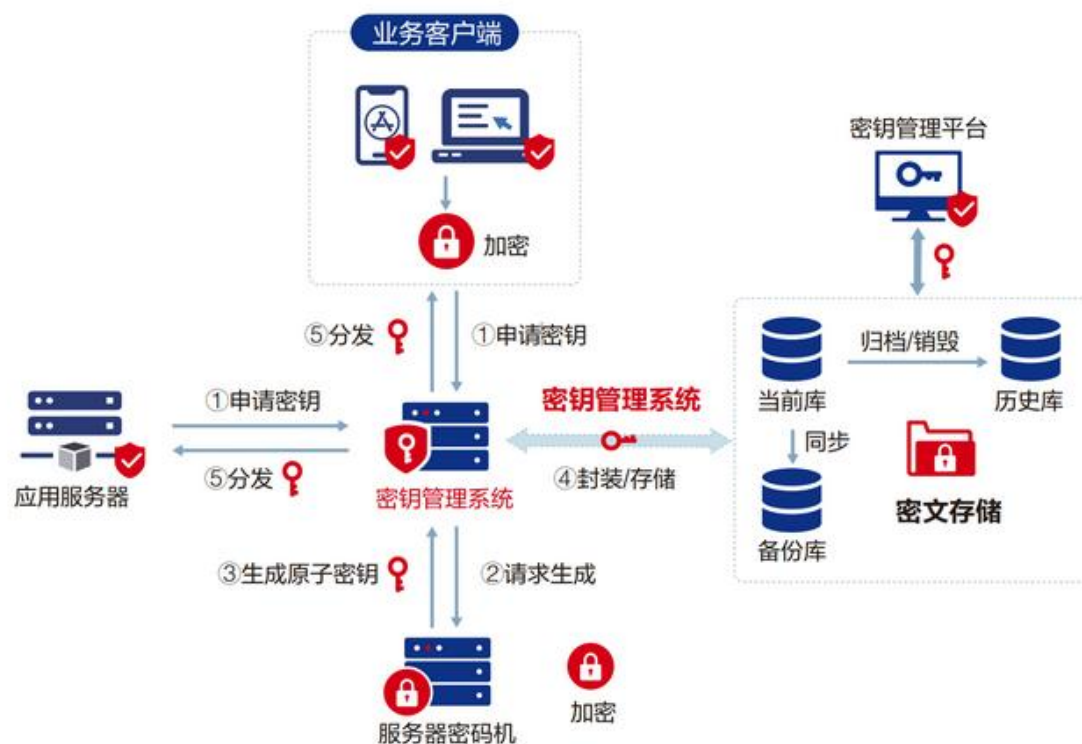


# 密钥管理

## 5.2.11 密钥管理系统

B U P T

密钥管理系统是为密码应用系统提供密钥生成、分发、存储、更新、归档、备份、恢复和销毁等全生命周期密钥管理能力，满足业务系统对密钥应用的管理需求。



5.3

# 密钥协商协议



# 密钥协商协议

B U P T



- ✓ 5.3.1 背景
- ✓ 5.3.2 初步设计
- ✓ 5.3.3 永久性的协议
- ✓ 5.3.4 认证约定
- ✓ 5.3.5 第一次协议改进
- ✓ 5.3.6 第二次协议改进
- ✓ 5.3.7 最终协议
- ✓ 5.3.8 不同角度看待协议
- ✓ 5.3.9 密钥的泄露
- ✓ 5.3.10 协议的复杂性
- ✓ 5.3.11 使用口令的密钥协商风险



## 密钥协商协议的基本背景

- Alice 和 Bob 希望通过密钥协商协议建立一个安全的会话密钥  $k$  。
- 会话密钥用于后续安全通信，保护数据交换过程。
- 身份认证的重要性：
  - 协商前必须确认对方身份。
  - 可以通过 RSA 签名、公钥基础设施(PKI)、共享密钥加 MAC 函数实现。





## 为什么已有共享密钥还要协商

### □原因一：

#### □消除密钥泄露风险：

□如果长期共享密钥泄露，之前协商出来的会话密钥仍然安全。

□密钥协商协议能提升系统的健壮性，即使密钥丢失，旧数据也被保护。

### □原因二：

#### □简化用户操作：

□共享密钥一般较脆弱（如短口令），而协商后的会话密钥更强大。

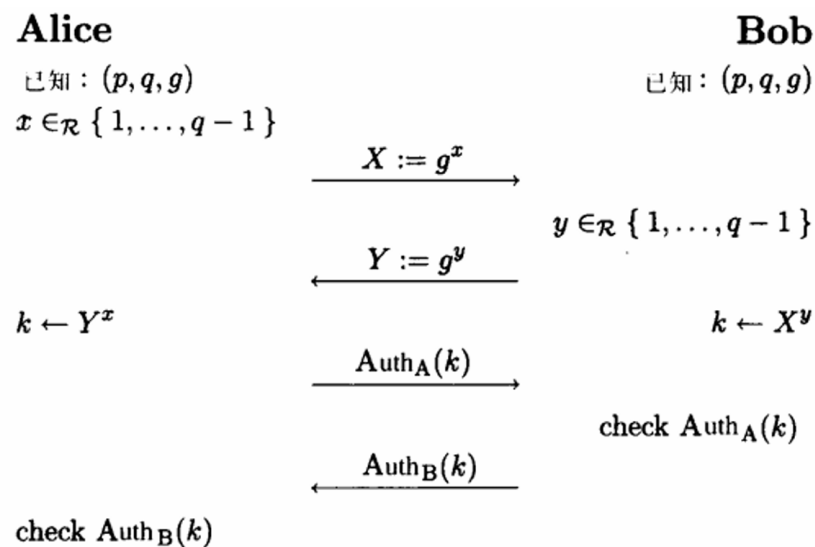


# 密钥协商协议 5.3.2 初步设计

B U P T

## 初步设计——简单的密钥协商

- Alice 和 Bob 通过 Diffie-Hellman (DH) 协议建立会话密钥  $k$ 。
- 双方各自发送一条消息，计算共享密钥。
- Alice 和 Bob 交换认证值 (Auth)，用于确认彼此计算出的密钥  $k$  是匹配的。



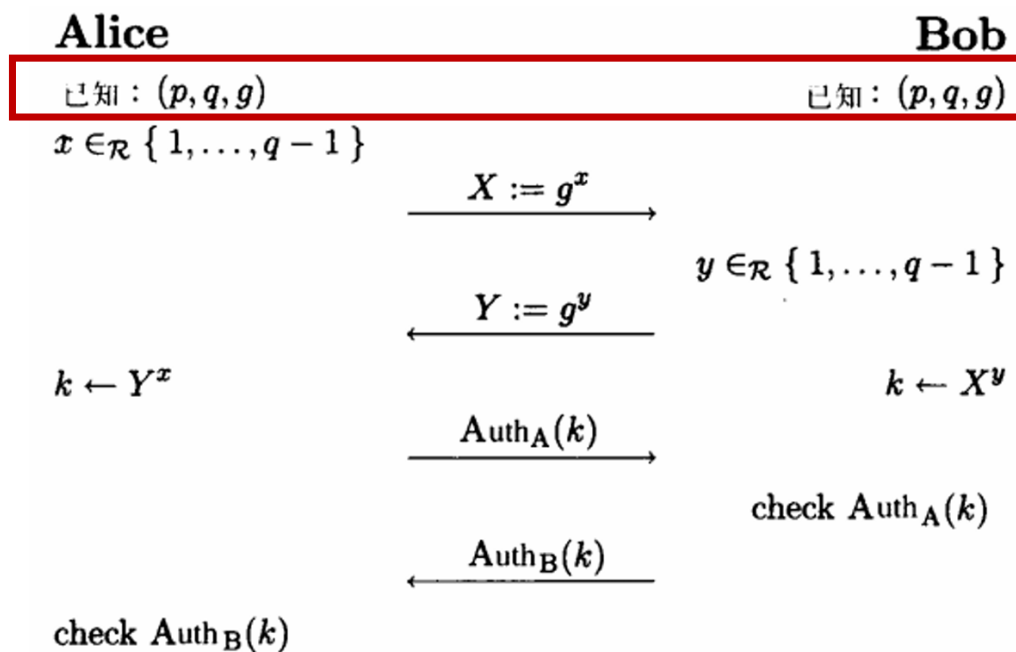


# 密钥协商协议 5.3.2 初步设计(2)

B U P T

## 存在的问题

□问题1：参数假设过于理想化。协议假设Alice和Bob事先知道  $(p, q, g)$ ，但在实际中这些值应当由协议确定。



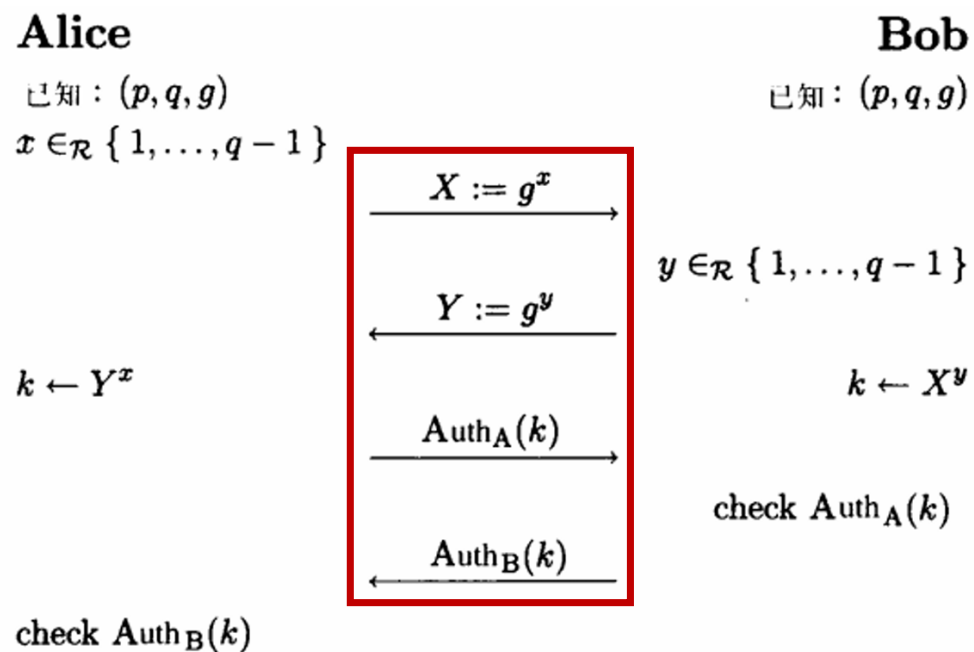


# 密钥协商协议 5.3.2 初步设计(3)

B U P T

## 存在的问题

□问题2：协议使用4条消息，然而只使用3条消息就可能达到目的了。



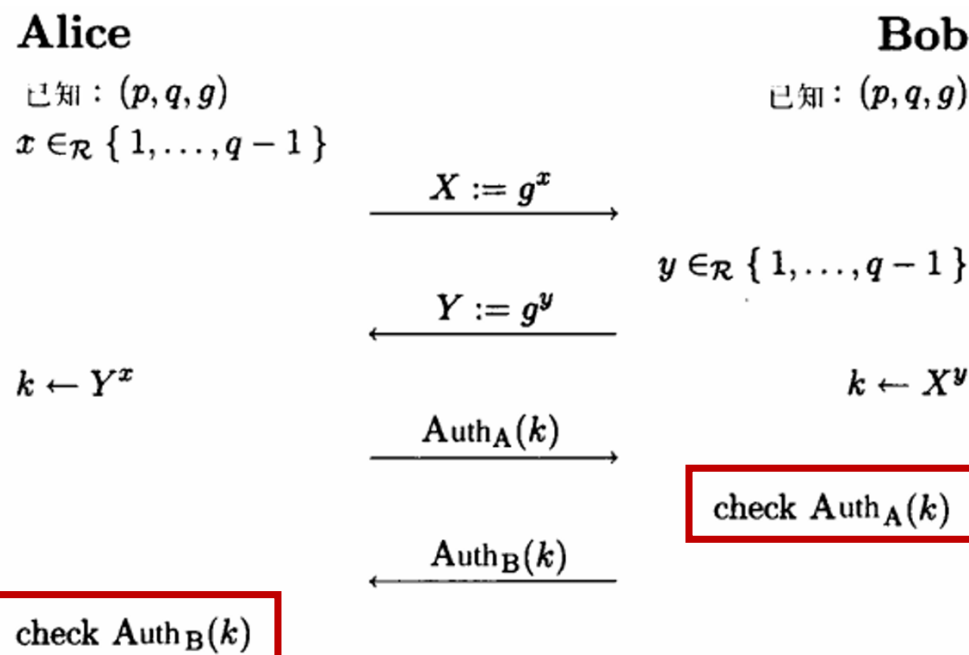


# 密钥协商协议 5.3.2 初步设计(4)

B U P T

## 存在的问题

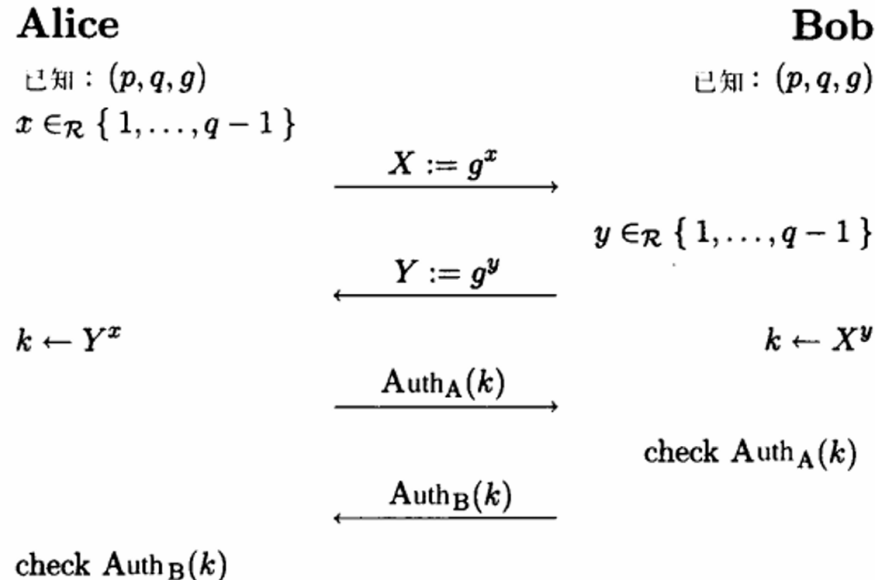
□问题3：认证值与密钥绑定问题。使用会话密钥  $k$  作为认证值的输入，若  $k$  泄露，认证值也可能暴露，影响安全。





## 存在的问题

- **问题4：** 消息结构存在隐患。认证消息之间可能存在相关性（如都由同一个MAC函数生成）。
- 如果攻击者拿到一个认证消息，有可能推测出另一个认证消息，导致协议被绕过。
- Alice**
**Bob**



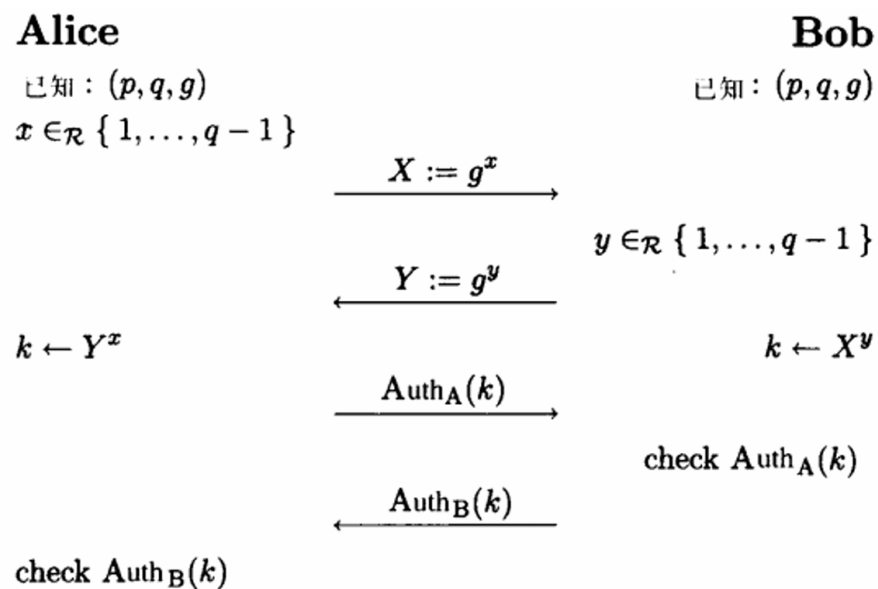


# 密钥协商协议 5.3.2 初步设计(6)

B U P T

## 存在的问题

□问题5：除非认证消息已经交换，否则不能使用密钥  $k$ 。这不是一个特别严重的问题，但在实际执行中仍存在风险，导致鲁棒性较差。





# 密钥协商协议

## 5.3.3 永久性的协议

B U P T

### 协议需要具备的持久性

- 协议需要长期运行且支持不同版本之间的兼容。
- 修改协议版本时，要保持新旧版本之间的互操作性，避免系统出错。
- 攻击者的利用手段：
  - 旧版协议漏洞会成为攻击点，攻击者可能会尝试强迫你使用旧版本带有漏洞下协议。
  - 协议升级时必须尽可能避免因版本回退而带来的安全风险。





### 协议升级的挑战

- 随着时间推移，未来需求变化、攻击手段演进，协议必须不断演化。
- 协议的生命周期设计要考虑抵御未来攻击，而不是仅仅满足当前需求。
- 协议复杂度带来的问题：
  - 新版协议越复杂，维护和更新难度越高。

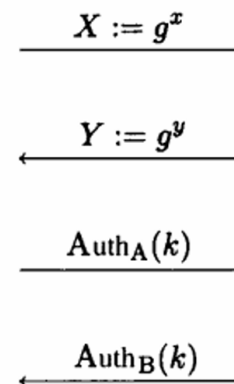


### 为什么要有认证约定？

- 协议中包含多种数据元素，必须确定哪些需要认证，防止遗漏。
- 简单有效的认证约定可以减少出错，提升安全性。
- 改进方法：
  - 每次发送认证消息时，包含之前所有交互过的重要数据。



check  $\text{Auth}_A(k)$





## 密钥协商协议 5.3.5 第一次协议改进

B U P T

- 避免使用固定的DH参数，提升协议的长期安全性。
- Alice选择参数  $(p, q, g)$ ，连同  $X$  和认证值  $Auth_A$  一起发送给Bob。

**Alice**

Choose suitable  $(p, q, g)$

$x \in_{\mathcal{R}} \{1, \dots, q-1\}$

$(p, q, g), X := g^x,$   
 $Auth_A$

**Bob**

Check  $(p, q, g), X, Auth_A$

$y \in_{\mathcal{R}} \{1, \dots, q-1\}$

$Y := g^y, Auth_B$

Check  $Y, Auth_B$

$k \leftarrow Y^x$

$k \leftarrow X^y$

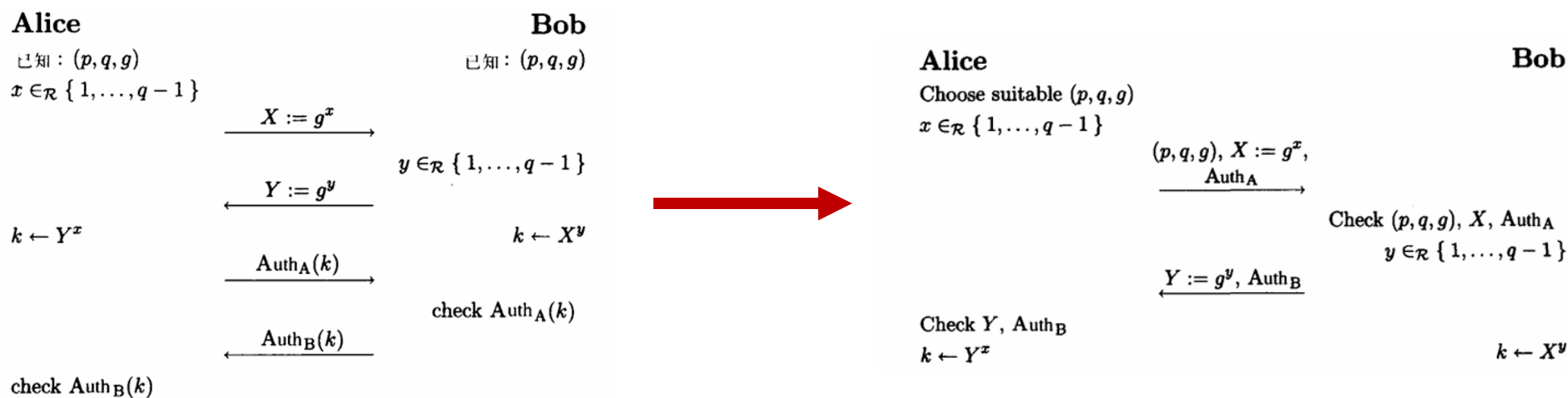


# 密钥协商协议 5.3.5 第一次协议改进(2)

B U P T

## 与第一次尝试相比的变化

- 将四条消息压缩为两条，提高了效率。
- 认证值一起发送，减少了交互次数。
- 检查机制更完善，保证数据的有效性。





## 面临的新问题

### □问题1：参数选择的安全性

- Bob 如果对 Alice 选择的DH参数不满意，必须中止协议并重新开始。
- 这会导致协议流程变复杂，增加了通信负担。

### □问题2：认证不足导致身份确认问题

- Bob 无法确认发送消息的人是否真的是 Alice。

### □传统解决方案：

- 在 Alice 的认证数据中包含一个由 Bob 挑选的随机数，确保 Alice 确实是实时响应的。



# 密钥协商协议

## 5.3.6 第二次协议改进

B U P T

### 改进动机：

□ 不由 Alice 选择 DH 参数，而是 Alice 只发送最小要求给 Bob，Bob 根据要求自己生成合适的 DH 参数。

### □ 新的流程特点：

- 消息总量增至 3 条，但安全性和灵活性提高。
- 认证过程仍保障每条消息的正确性。



# 密钥协商协议 5.3.6 第二次协议改进(2)

B U P T

## 协议执行步骤

- Alice 发送参数最小长度  $s$  和随机数  $N_a$ 。
- Bob 选择  $(p, q, g)$ 、生成  $X$ ，并附上  $Auth_B$  返回给 Alice。
- Alice 验证参数及  $Auth_B$ ，发送  $Y$  和  $Auth_A$ 。
- 双方各自计算共享密钥  $k$ 。

**Alice**

$s \leftarrow \min p \text{ size}$   
 $N_a \in_{\mathcal{R}} 0, \dots, 2^{256} - 1$

Check  $(p, q, g), X, Auth_B$   
 $y \in_{\mathcal{R}} \{1, \dots, q-1\}$

$k \leftarrow X^y$

**Bob**

Choose  $(p, q, g)$   
 $x \in_{\mathcal{R}} \{1, \dots, q-1\}$

$(p, q, g), X := g^x,$   
 $Auth_B$

$Y := g^y, Auth_A$

Check  $Y, Auth_A$   
 $k \leftarrow Y^x$





### 面临的新问题

#### □代数结构的安全隐患：

- 最终生成的密钥  $k$  具有代数结构，可能被数学方法利用攻击。
- 过去20年，很多公钥系统因代数结构问题被破解。

#### □对策：

- 对最终密钥  $k$  进行Hash压缩，使其变为固定长度的随机数，消除代数痕迹。



# 密钥协商协议 5.3.7 最终协议

B U P T

**Alice**

$s \leftarrow \text{min } p \text{ size}$

$N_a \in_{\mathcal{R}} 0, \dots, 2^{256} - 1$

$\xrightarrow{s, N_a}$

**Bob**

Choose  $(p, q, g)$

$x \in_{\mathcal{R}} \{1, \dots, q - 1\}$

$(p, q, g), X := g^x,$

$\xleftarrow{\text{Auth}_B}$

Check  $(p, q, g), X, \text{Auth}_B$

$y \in_{\mathcal{R}} \{1, \dots, q - 1\}$

$\xrightarrow{Y := g^y, \text{Auth}_A}$

Check  $Y, \text{Auth}_A$

$k \leftarrow \text{SHA}_{d-256}(X^y)$

$k \leftarrow \text{SHA}_{d-256}(Y^x)$



### 协议的最终优化

- 缓存合理的 DH 参数集，减少每次重新生成和传输的开销。
- 应用程序可以选择固定 DH 参数，简化处理，但需谨慎。
- 注意事项：
  - 简单优化容易引入新漏洞，必须非常小心。
  - 协议设计没有简单万能的检测规则，它既是科学也是艺术。



### 协议应该具备的特性

- 能让通信双方彼此信任。
  - 能抵御重放攻击和伪造消息。
  - 能避免因参数问题导致安全隐患。
- 
- 从不同参与方（Alice、Bob、攻击者）视角观察，能更全面理解协议设计。



# 密钥协商协议 5.3.8 不同角度看待协议(2)

B U P T

## 从Alice的视角观察协议

- Alice 确认收到的消息确实来自 Bob , 因为包含她生成的随机数  $N_a$  。
- Alice 验证 DH 参数的正确性, 确保密钥  $k$  是安全推导出来的。
- 确认只有自己和 Bob 知道最终共享密钥  $k$  。

**Alice**

$s \leftarrow \text{min } p \text{ size}$   
 $N_a \in_{\mathcal{R}} 0, \dots, 2^{256} - 1$

**Bob**

$\xrightarrow{s, N_a}$

Choose  $(p, q, g)$   
 $x \in_{\mathcal{R}} \{1, \dots, q-1\}$

$(p, q, g), X := g^x,$   
 $\xleftarrow{\text{Auth}_B}$

Check  $(p, q, g), X, \text{Auth}_B$   
 $y \in_{\mathcal{R}} \{1, \dots, q-1\}$

$\xrightarrow{Y := g^y, \text{Auth}_A}$

$k \leftarrow \text{SHA}_{d-256}(X^y)$

Check  $Y, \text{Auth}_A$   
 $k \leftarrow \text{SHA}_{d-256}(Y^x)$

Alice通过认证和验证机制, 确信通信过程的安全可靠。

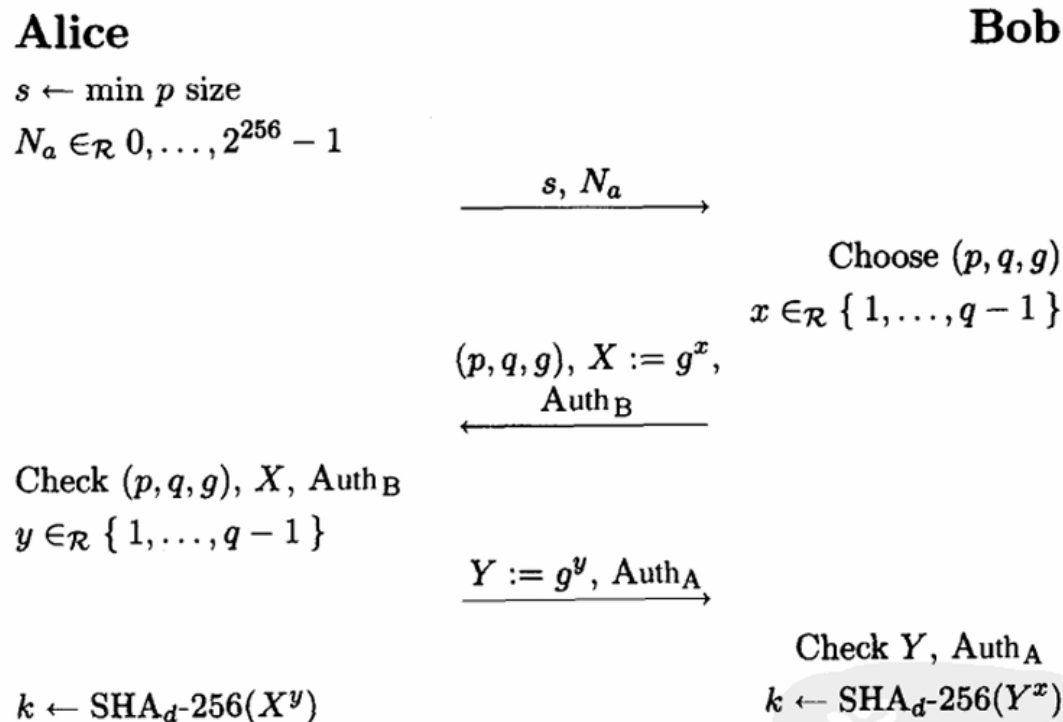


# 密钥协商协议 5.3.8 不同角度看待协议(3)

B U P T

## 从Bob的视角观察协议

- Bob 收到的第一条消息几乎无有用信息，第二条消息起关键作用。
- Bob 通过认证过程确信消息来自 Alice，并确认 DH 参数的安全性。
- Bob 知道，只有他和 Alice 能够计算出最终的共享密钥  $k$ 。



Bob也能确认通信对象和密钥的正确性，从而安全完成协商。

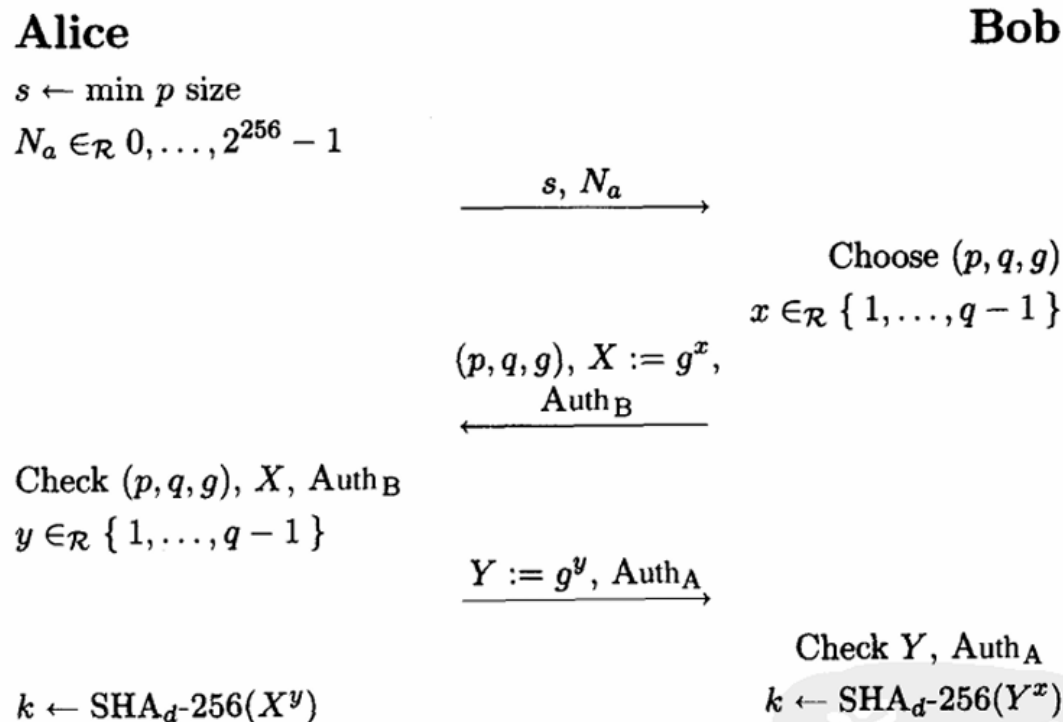


# 密钥协商协议 5.3.8 不同角度看待协议(4)

B U P T

## 从攻击者的视角观察协议

- 即使能监听到所有通信内容，攻击者仍无法推导出任何密钥。
- 需要尝试主动攻击，比如伪造、篡改或插入消息，但协议设计使这种攻击无效。
- 随机数的引入阻断了重放攻击的可能。



协议抵抗了监听、篡改和重放三类典型威胁。



### 密钥泄露会带来什么后果？

- 即使系统的其他部分被攻击，只要攻击者不知道密钥，仍无法获取通信内容；
- 若 Alice 丢失了会话密钥
  - 若攻击者不知情，只需与 Bob 重新协商新的会话密钥；
- 若 Alice 丢失了认证密钥
  - 若攻击者不知情，她仍可继续安全通信；
  - 攻击者若得知认证密钥，风险才真正开始——此时攻击者可冒充 Alice 发起新会话，直到 Bob 停止信任所有以 Alice 名义发的消息。





### 协议设计的关键保障机制

- 密钥泄露是现实中重要的风险，协议必须确保通信“前向保密性”：
  - 已完成的通信无法被解密；
  - 即便认证密钥泄露，历史通信仍是安全的；
- 协议的一个核心目标：
  - 即使攻击者知道认证密钥，也不能重建过去通信内容。



# 密钥协商协议

## 5.3.9 密钥的泄露(3)

B U P T

### 会话密钥泄露的保护策略

- 协议中的会话密钥  $k = g^{xy}$  是由随机值计算所得；
- 即使泄露，也不暴露 Alice 和 Bob 的认证密钥；
- 不同会话生成的密钥  $k$  是相互独立的；
- 总结：我们的协议对密钥泄露提供了“可能的最好保护”。



# 密钥协商协议 5.3.10 协议的复杂性

B U P T

## 协议运行中有哪些计算？

□ 假设 DH 参数的选择与验证都已缓存，计算复杂度集中体现在：

□ Alice 与 Bob 需执行 3 次 DH 指数运算：

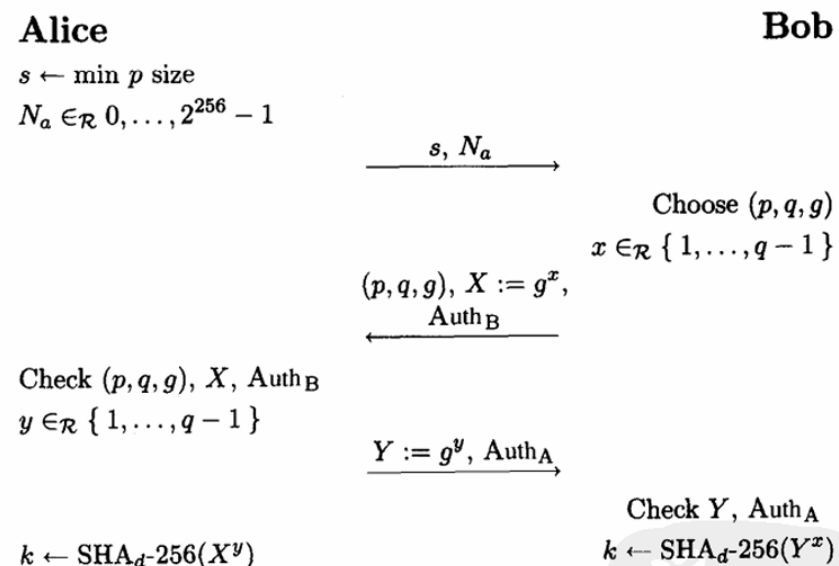
□ 一次认证生成；

□ 一次认证验证；

□ 一次用于最终密钥生成；

□ 其他操作开销较小，可以忽略不计：

□ 如随机数生成、Hash 运算等。





### 为什么协议设计是极具挑战性的？

- 即使一个看似简单的协议也可能迅速扩展成数十页的规范文档。
- 要全面描述所有细节（如DH参数生成、验证方式等），往往极其困难，甚至不可能一次性完成。
- 一套协议文档甚至可达上百页，这远超人类大脑轻松理解的范围，容易引入安全隐患。
- 如果设计者本人都难以完全理解协议，漏洞几乎是不可避免的。



### 协议设计从不简单，请保持警惕

- 协议看起来容易理解，但这是表象。协议设计极其复杂，需要丰富经验才能避免常见陷阱。
- 很多协议看似可行，实则暗藏漏洞。我们所设计的密钥协商协议，也可能出错。
- 不要低估协议设计的难度。经验和反思才是安全的保障。



## 密钥协商协议 5.3.11 使用口令的密钥协商风险

B U P T

- 有些协议使用用户口令作为密钥认证的基础。
- 风险在于：
  - 通信记录可能被监听，攻击者可用字典爆破猜测正确口令。
- 理想协议应能防止攻击者“离线”暴力破解。
- 谨慎采用基于口令的密钥协商协议，除非有充分理由与保障。



# 课堂思考

---

B U P T

**2.如果攻击者已经掌握了某一方的认证密钥，我们能从协议层面做什么来减少损失？**



# 课后作业

B U P T

- 1.简述 SM9 国密算法的基本原理、核心组成部分及其典型应用场景。
- 2.协议的设计动机会如何影响协议结构？请举一个例子来说明。



**谢谢大家**

---